

PERFORMANCE ENGINEERING

Lab 5

Jort Oosterveld S5545781 & Sergiu Preda S6778542
Group 11

October 7, 2026

1 INFORMATION EXCHANGE ANALYSIS OF A CACHE-BLOCKED MATRIX-VECTOR MULTIPLY

The implementation of the cache-blocked matrix-vector multiplication can be seen in listing 1.

```
1 /*
2  * y = A * x using cache blocking
3  * N: Matrix dimension (N x N)
4  * B: Block size (N % B = 0)
5  * A: Flattened N x N matrix
6  * x: Input vector of size N
7  * y: Output vector of size N
8  */
9 void blocked_mat_vec(int N, int B, const double *A, const double *x, double *y)
10 {
11     // 1. Initialize output vector
12     for (int i = 0; i < N; i++) {
13         y[i] = 0.0;
14     }
15
16     // 2. Iterate over blocks
17     for (int i = 0; i < N; i += B) {          // Row blocks
18         for (int j = 0; j < N; j += B) {      // Column blocks
19
20             // 3. Compute multiplication for the current B x B block
21             for (int ii = i; ii < i + B; ii++) {
22                 for (int jj = j; jj < j + B; jj++) {
23                     y[ii] += A[ii * N + jj] * x[jj];
24                 }
25             }
26         }
27     }
28 }
```

Listing 1: Matrix-vector multiplication: cache-blocked implementation.

To calculate I_{mem} for a matrix-vector multiplication of the form $\vec{y} = A \cdot \vec{x}$, we assume:

- The entire vector $\vec{x}$ cannot be kept in cache.
- A fraction N/B of the vector can stay in cache.
- The block size B satisfies $N \% B = 0$.

This means there is still one load for every element of A , thus $8N^2$ bytes. There is one load for every element in each block, thus $8 \cdot (B \cdot N/B) = 8N$. There is one load, and one store for every partial sum of the row (each block), thus $2 \cdot 8 \cdot N \cdot N/B = 16 \cdot N^2/B$. The total number of stores and loads then adds up to:

$$I_{mem} = 8N^2 + 16N^2/B + 8N$$

2 CACHEGRIND

The cache-blocked routine of Listing 1 was run under Cachegrind (`valgrind -tool=cachegrind`) on Sergiu’s machine (see Section 4), which has a 48 KiB L1D cache per core. We selected $N = 8192$: with double-precision numbers (8 bytes per element) a single vector of size N occupies $8192 \times 8 = 65,536$ bytes (64 KiB), which strictly exceeds the L1D cache capacity.

The bash script in Listing 2 executes the program for the block sizes $B = 8, 16, 32, \dots, 1024$ while keeping $N = 8192$ constant and extracts the D1 misses (L1 data cache misses) from Cachegrind’s output.

```

1 #!/bin/bash
2
3 # Fixed matrix size (N=8192 ensures the vector exceeds the 48 KiB L1d cache)
4 N=8192
5
6 # Array of block sizes to test
7 BLOCK_SIZES=(8 16 32 64 128 256 512 1024)
8
9 # Output CSV file
10 OUTPUT_FILE="cache_misses.csv"
11
12 # Initialize the CSV with headers
13 echo "BlockSize_B,D1_Misses" > "$OUTPUT_FILE"
14 echo "Starting Cachegrind benchmark for N=$N..."
15
16 for B in "${BLOCK_SIZES[@]"; do
17     echo -n "Testing B=$B... "
18
19     # --cachegrind-out-file=/dev/null prevents the .out files from being
20     # generated
21     # awk '{print $4}' grabs the actual number instead of the word "misses:"
22     MISSES=$(valgrind --tool=cachegrind --cachegrind-out-file=/dev/null ./
23 blocked_matvec $N $B 2>&1 | grep "D1 misses:" | awk '{print $4}' | tr -d ',
24 ')
25
26 # Append the result to the CSV
27 echo "$B,$MISSES" >> "$OUTPUT_FILE"
28 echo "Misses: $MISSES"
29 done
30 echo "Done! Results saved to $OUTPUT_FILE."
```

Listing 2: Script running the blocked matrix-vector multiply under Cachegrind for a range of block sizes.

The results are listed in Table 1 and visualized in Figure 1.

The graph shows that the number of cache misses peaks at small block sizes (e.g., $B = 16$) and drops significantly as the block size increases up to $B = 1024$. This indicates better utilization of the L1D cache capacity before eviction becomes strictly necessary.

Block size B	D1 cache misses
8	17839350
16	21525718
32	19170454
64	17992726
128	17403671
256	17108263
512	16945895
1024	16863598

Table 1: L1D cache misses for different block sizes with $N = 8192$ (large enough that the vector does not fit in L1D), measured on Sergiu’s system, see Section 4.

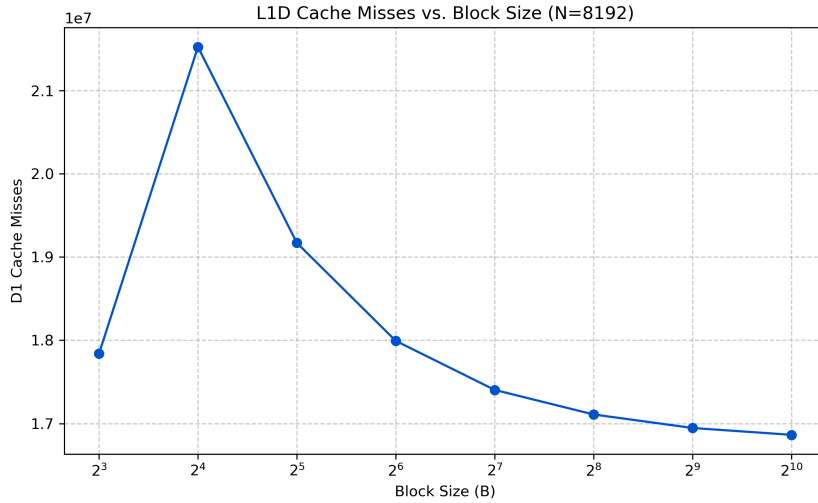


Figure 1: L1D cache misses as a function of block size B for $N = 8192$.

3 MEMORY ALLOCATION OVERHEAD

To measure the overhead of allocating and freeing memory dynamically we wrote the C++ benchmark of Listing 3. It uses `clock_gettime(CLOCK_MONOTONIC)` to measure the time of a malloc and free pair for different sizes. It does this in two ways, for consistency:

1. **Batch timing:** the timer is read once, $1e6$ malloc/free pairs are executed, the timer is read again and the elapsed time is divided by the number of pairs.
2. **Per-pair timing:** every single pair is wrapped in two timer reads, so that the full distribution (mean, median, standard deviation, minimum, maximum) of $1e6$ samples is obtained. The cost of the timer itself is measured separately and subtracted.

The allocated block is touched with a `volatile` store before it is freed, because otherwise the compiler can remove the dead code. Every size is preceded by an unmeasured warm-up pass of 10^5 pairs to remove any one-time costs.

```

1 static const clockid_t CLOCK_ID = CLOCK_MONOTONIC;
2
3 static inline long long timespec_diff_ns(const struct timespec *t2,
4                                         const struct timespec *t1)
5 {

```

```

6     return ((long long)(t2->tv_sec - t1->tv_sec) * 1000000000LL) +
7           (t2->tv_nsec - t1->tv_nsec);
8 }
9
10 /* running mean / variance, min, max and median
11  * of all samples added with add(). */
12 struct TimingStats { /* ... */ void add(long long ns); double mean; /* ... */ };
13
14 /* Cost of two back-to-back clock_gettime() calls */
15 static TimingStats measure_timer_overhead(int iterations)
16 {
17     TimingStats stats;
18     struct timespec t1, t2;
19     for (int i = 0; i < iterations; ++i) {
20         clock_gettime(CLOCK_ID, &t1);
21         clock_gettime(CLOCK_ID, &t2);
22         stats.add(timespec_diff_ns(&t2, &t1));
23     }
24     return stats;
25 }
26
27 /* individual pair timing */
28 static TimingStats measure_alloc_per_pair(size_t alloc_size, int iterations)
29 {
30     TimingStats stats;
31     struct timespec start, end;
32     for (int i = 0; i < iterations; i++) {
33         clock_gettime(CLOCK_ID, &start);
34         void *ptr = malloc(alloc_size);
35         // touch the memory so the allocation cannot be optimised away
36         if (ptr != NULL) {
37             ((volatile char *)ptr)[0] = 'a';
38             free(ptr);
39         }
40         clock_gettime(CLOCK_ID, &end);
41         stats.add(timespec_diff_ns(&end, &start));
42     }
43     return stats;
44 }
45
46 /* batch timing, clock overhead is insignificant */
47 static double measure_alloc_batch(size_t alloc_size, int iterations)
48 {
49     struct timespec start, end;
50     clock_gettime(CLOCK_ID, &start);
51     for (int i = 0; i < iterations; i++) {
52         void *ptr = malloc(alloc_size);
53         if (ptr != NULL) {
54             ((volatile char *)ptr)[0] = 'a';
55             free(ptr);
56         }
57     }
58     clock_gettime(CLOCK_ID, &end);
59     return static_cast<double>(timespec_diff_ns(&end, &start)) / iterations;
60 }
61
62 int main()
63 {
64     const int iterations = 1000000;
65     const size_t sizes[] = {8, 64, 256, 1024, 4096, 65536, 1048576};
66
67     TimingStats timer = measure_timer_overhead(iterations);
68     const double timer_overhead_ns = timer.median_ns();

```

```

69
70     for (size_t size : sizes) {
71         measure_alloc_batch(size, iterations / 10); // warmup
72
73         double batch_ns = measure_alloc_batch(size, iterations);
74         TimingStats pp = measure_alloc_per_pair(size, iterations);
75
76         double mean_ns    = pp.mean          - timer_overhead_ns;
77         double median_ns  = pp.median_ns() - timer_overhead_ns;
78     }
79     return 0;
80 }

```

Listing 3: Benchmark for the overhead of a `malloc/free` pair (`benchmark_malloc.cpp`, statistics bookkeeping omitted).

JUSTIFICATION OF THE APPROACH

A single `malloc/free` pair takes only a few nanoseconds, which is less than the cost of reading the clock. Measuring the timer gives a median of 20 ns and a minimum of 10 ns. The timer also records in steps of 10ns in these measurements, even though it should have a resolution of 1ns. This means the resolution is not high enough for individual pair measurements.

The batch approach is therefore leading in these results, and the individual measurements are included for completeness. It makes the clock overhead insignificant, and through the large sample size, it is more accurate.

RESULTS

The benchmark was compiled with `g++ -O2 (glibc 2.39)` and executed on Jort’s machine. The results are listed in Table 2 and plotted in Figure 2.

Size (bytes)	Batch (ns / pair)	Per-pair raw (ns / pair)	Per-pair mean (ns / pair)	Per-pair median (ns / pair)	Std. dev. (ns)
8	8.14	23.93	3.93	0	21.81
64	7.87	23.84	3.84	0	36.95
256	7.91	24.03	4.03	0	42.89
1024	7.92	23.85	3.85	0	21.17
4096	17.22	29.57	9.57	10	26.32
65536	17.53	30.30	10.30	10	24.47
1048576	17.71	30.18	10.18	10	29.02

Table 2: Time per `malloc/free` pair for different allocation sizes, 10^6 pairs per size. “Per-pair raw” includes the timer overhead; “per-pair mean” and “per-pair median” have the 20 ns median timer overhead subtracted.

In figure 2 we see that there are two levels of latency for the `malloc/free` pairs. The lower one ($\approx 8ns$) exists likely because of the thread cache, and the higher level ($\approx 17ns$) exists because the thread cache is exceeded, but the warmup already raised the threshold for larger allocations, which makes this level also stay somewhat constant.

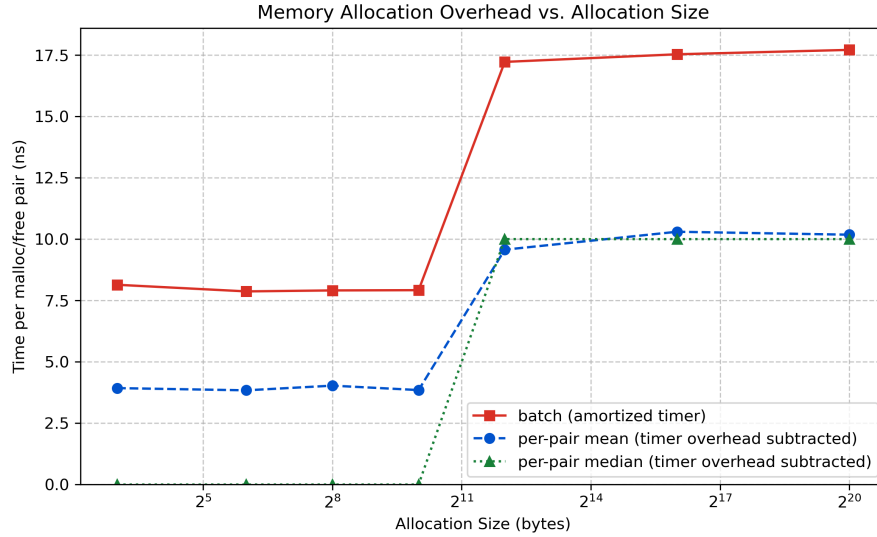


Figure 2: Time per malloc/free pair versus allocation size. Batch measurement takes the average of all iterations, and per-pair statistics have the timer overhead subtracted.

4 HARDWARE INFO

4.1 JORT OOSTERVELD

Type of computer: Laptop

Operating system: Ubuntu 24.04.4 LTS

Kernel version: Linux 7.0.0-38-generic

Hardware Model: Acer Aspire AG15-42P

Graphics: AMD Radeon Graphics

```

1 description: Computer
2 width: 64 bits
3 capabilities: smp vsyscall32
4 *-core
5     description: Motherboard
6     physical id: 0
7 *-memory
8     description: System memory
9     physical id: 0
10    size: 15GiB
11 *-cpu
12     product: AMD Ryzen 7 5825U with Radeon Graphics
13     vendor: Advanced Micro Devices [AMD]
14     physical id: 1
15     bus info: cpu@0
16     version: 25.80.0
17     size: 3629MHz
18     capacity: 4547MHz
19     width: 64 bits

```

Listing 4: From lshw

```

1 $ lscpu --cache
2
3 NAME ONE-SIZE ALL-SIZE WAYS TYPE LEVEL SETS PHY-LINE COHERENCY-SIZE
4 L1d 32K 256K 8 Data 1 64 1 64
5 L1i 32K 256K 8 Instruction 1 64 1 64

```

6	L2	512K	4M	8	Unified	2	1024	1	64
7	L3	16M	16M	16	Unified	3	16384	1	64

Listing 5: From lscpu

4.2 SERGIU PREDA

Type of computer: Laptop

Operating system: Windows 11 with WSL Ubuntu 22.04 LTS

Kernel version: Linux 6.18.33.2-microsoft-standard-WSL2

Hardware Model: ASUS ROG TUF F15

Memory: 7.6GiB

```

1 Model name:          12th Gen Intel(R) Core(TM) i7-12700H
2 Flags:              ... avx2 ...

```

Listing 6: From lscpu

```

1 $ lscpu --cache
2 NAME ONE-SIZE ALL-SIZE WAYS TYPE          LEVEL  SETS PHY-LINE COHERENCY-SIZE
3 L1d      48K      480K   12 Data             1     64      1          64
4 L1i      32K      320K    8 Instruction        1     64      1          64
5 L2      1.3M     12.5M   10 Unified           2   2048      1          64
6 L3       24M      24M   12 Unified           3 32768      1          64

```

Listing 7: From lscpu --cache

For additional microarchitecture information, the following sources were used:

- uops.info instruction latency and throughput database: <https://uops.info/>
- Agner Fog instruction tables: https://www.agner.org/optimize/instruction_tables.pdf